

Documentación Técnica del Proyecto

1. ¿De qué trata este proyecto?

Este proyecto es una API REST construida con Node.js y TypeScript puro, sin usar ningún framework externo como Express. Básicamente, lo que hace es exponer endpoints HTTP (las URLs a las que le puedes hacer peticiones desde Postman) para gestionar dos recursos principales: Productos y Clientes.

Además de funcionar como API, el mismo proyecto también tiene un menú interactivo de consola, por lo que al ejecutar el proyecto se va a ejecutar el http y el menú a la vez

La estructura del proyecto sigue una arquitectura por capas (o módulos), donde cada capa tiene su responsabilidad bien definida:

Capa de Entrada (API): El router y el server, que reciben y responden las peticiones HTTP.

Capa de Negocio (Services): Donde está toda la lógica de lo que se puede hacer con los datos.

Capa de Datos (Repository): Quien realmente toca los archivos JSON donde se guardan los datos.

Capa de Errores: Clases especiales para identificar exactamente qué tipo de error ocurrió.

Capa de Validación: Revisa que los datos que manda el usuario estén bien antes de guardarlos.

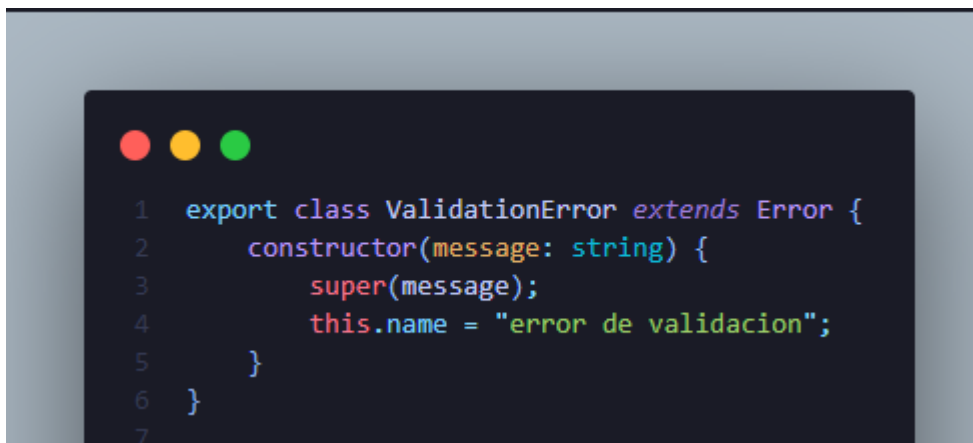
2. Los Errores Personalizados (customErrors.ts)

Archivo: src/errors/customErrors.ts

Este fue uno de los primeros archivos que se creó porque es la base de todo el manejo de errores del proyecto. La idea es que en lugar de lanzar un Error genérico que diga solo "algo salió mal", tengamos errores específicos que digan exactamente qué tipo de problema ocurrió. Se crearon tres clases de error:

2.1 ValidationError

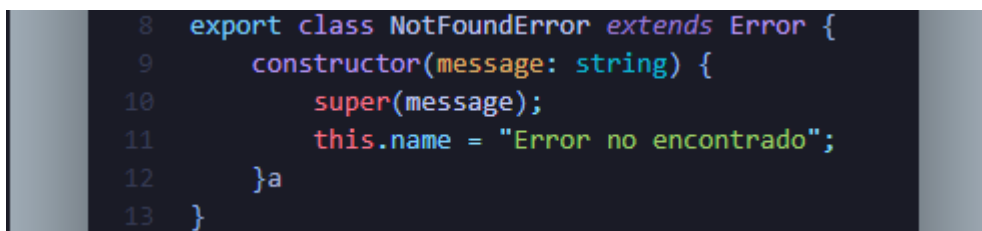
Se lanza cuando el usuario manda datos que no cumplen con los requisitos. Por ejemplo, si el nombre del producto viene vacío, o si el correo del cliente no tiene el formato correcto.



```
1  export class ValidationError extends Error {  
2      constructor(message: string) {  
3          super(message);  
4          this.name = "error de validacion";  
5      }  
6  }  
7
```

2.2 NotFoundError

Se lanza cuando se intenta buscar, actualizar o eliminar algo que no existe. Por ejemplo, si se hace un GET /productos/999 y no hay ningún producto con ese ID.



```
8  export class NotFoundError extends Error {  
9      constructor(message: string) {  
10         super(message);  
11         this.name = "Error no encontrado";  
12     }  
13 }
```

2.3 DatabaseError

Se lanza cuando hay un problema al leer o escribir los archivos JSON que funcionan como base de datos. Por ejemplo, si el archivo está corrupto o no tiene permisos de escritura.

```
15 export class DatabaseError extends Error {  
16     constructor(message: string) {  
17         super(message);  
18         this.name = "Error de DB";  
19     }  
20 }  
21
```

3. El Patrón Repositorio (clienteRepository.ts / productoRepository.ts)

Archivos: src/data/clienteRepository.ts y src/data/productoRepository.ts

El patrón repositorio es básicamente una capa intermedia entre los servicios y los archivos JSON. En lugar de que los servicios lean y escriban directamente en los archivos, le piden al repositorio que lo haga.

La diferencia con los archivos reader.ts y writer.ts (que ya existían) es que el repositorio sabe qué tipo de datos maneja (Clientes o Productos), mientras que reader/writer son funciones genéricas de bajo nivel que solo abren y cierran archivos sin saber qué contienen.

3.1 ClienteRepository

Tiene tres métodos principales:

```

1 import { Cliente } from "../models/clientes";
2 import { leerClientes } from "../utils/reader";
3 import { escribirClientes } from "../utils/writer";
4 import { DatabaseError } from "../errors/customErrors";
5
6 export class ClienteRepository {
7     async obtenerTodos(): Promise<Cliente[]> {
8         try {
9             const data = await leerClientes();
10            if (typeof data === "string") {
11                return [];
12            }
13            return Array.isArray(data) ? data : [];
14        } catch (error: any) {
15            throw new DatabaseError("Error al obtener los clientes: " + error.message);
16        }
17    }
18
19    async buscarPorId(id: number): Promise<Cliente | null> {
20        const clientes = await this.obtenerTodos();
21        return clientes.find(c => c.id_cliente === id) || null;
22    }
23
24    async guardar(clientes: Cliente[]): Promise<void> {
25        try {
26            await escribirClientes(clientes);
27        } catch (error: any) {
28            throw new DatabaseError("Error al guardar los clientes: " + error.message);
29        }
30    }
31 }
32

```

3.2 ProductoRepository

Funciona exactamente igual que ClienteRepository pero para el archivo producto.json. Tiene los mismos tres métodos: obtenerTodos(), buscarPorId() y guardar().

```

1 import { Producto } from "../models/producto";
2 import { leerProductos } from "../utils/reader";
3 import { escribirProductos } from "../utils/writer";
4 import { DatabaseError } from "../errors/customErrors";
5
6 export class ProductoRepository {
7     async obtenerTodos(): Promise<Producto[]> {
8         try {
9             const data = await leerProductos();
10            if (typeof data === "string") {
11                return [];
12            }
13            return Array.isArray(data) ? data : [];
14        } catch (error: any) {
15            throw new DatabaseError("Error al obtener los productos: " + error.message);
16        }
17    }
18
19    async buscarPorId(id: number): Promise<Producto | null> {
20        const productos = await this.obtenerTodos();
21        return productos.find(p => p.id_producto === id) || null;
22    }
23
24    async guardar(productos: Producto[]): Promise<void> {
25        try {
26            await escribirProductos(productos);
27        } catch (error: any) {
28            throw new DatabaseError("Error al guardar los productos: " + error.message);
29        }
30    }
31 }
32

```

4. Las Validaciones (validaciones.ts)

Archivo: src/validator/validaciones.ts

Este archivo ya existía, pero se refactorizó para que en lugar de lanzar un Error genérico, lance el ValidationError personalizado.

4.1 validarCliente(c)

Revisa que el objeto Cliente tenga todos los campos correctos: nombre no vacío, apellido no vacío, teléfono válido mayor a 0, DPI válido mayor a 0, y correo perteneciente a @gmail.com, @yahoo.com o @outlook.com.

4.2 validarProducto(p)

Revisa que el objeto Producto tenga todo en orden: nombre no vacío, código de producto número válido mayor a 0, precio mayor a 0, y cantidad no negativa.

Código:

```
1 import { Cliente } from "../models/clientes";
2 import { Producto } from "../models/producto";
3 import { ValidationError } from "../errors/customErrors";
4
5 export function validarCliente(c: Cliente): void {
6   if (!c.nombre_cliente || c.nombre_cliente.trim() === "") {
7     throw new ValidationError("El nombre no puede estar vacío.");
8   }
9   if (!c.apellido_cliente || c.apellido_cliente.trim() === "") {
10    throw new ValidationError("El apellido no puede estar vacío.");
11  }
12  if (typeof c.telefono_cliente !== "number" || isNaN(c.telefono_cliente) || c.telefono_cliente <= 0) {
13    throw new ValidationError("El telefono debe ser un numero valido.");
14  }
15  if (typeof c.dpi_cliente !== "number" || isNaN(c.dpi_cliente) || c.dpi_cliente <= 0) {
16    throw new ValidationError("El DPI debe ser un numero valido.");
17  }
18
19  const correo = c.correo_cliente.toLowerCase();
20  if (!correo.includes("@gmail.com") && !correo.includes("@yahoo.com") && !correo.includes("@outlook.com")) {
21    throw new ValidationError("El correo debe ser de @gmail.com, @Yahoo.com o @outlook.com.");
22  }
23 }
24
25 export function validarProducto(p: Producto): void {
26   if (!p.nombre_producto || p.nombre_producto.trim() === "") {
27     throw new ValidationError("El nombre no puede estar vacío.");
28   }
29   if (typeof p.codigo_producto !== "number" || isNaN(p.codigo_producto) || p.codigo_producto <= 0) {
30     throw new ValidationError("El codigo debe ser un numero valido.");
31   }
32   if (typeof p.precio !== "number" || isNaN(p.precio) || p.precio <= 0) {
33     throw new ValidationError("El precio debe ser mayor a 0.");
34   }
35   if (typeof p.cantidad !== "number" || isNaN(p.cantidad) || p.cantidad < 0) {
36     throw new ValidationError("La cantidad no puede ser negativa.");
37   }
38 }
```

5. Los Servicios (productoService.ts / clientesService.ts)

Archivos: src/services/productoService.ts y src/services/clientesService.ts

Los servicios son la capa de negocio. Son los que deciden qué se puede y qué no se puede hacer. Se refactorizaron para usar el repositorio y se agregaron funciones nuevas para que la API funcione correctamente.

5.1 Funciones del productoService

listarProductos(): Devuelve la lista completa de productos como arreglo limpio.

buscarProducto(id): Busca un producto por ID. Si no lo encuentra, lanza un NotFoundError.

crearProducto(nuevoProducto): Para el menú de consola. Valida el objeto y lo guarda con ID auto-incrementado.

agregarProducto(...): Para el router HTTP. Implementa ID auto-incrementado si el ID no viene, es null, NaN, 0 o negativo:

typescript

```
let idAutoIncrement = id;
```

```
if (id === undefined || id === null || Number.isNaN(id) || id <= 0) {
```

```
    idAutoIncrement = productos.length > 0
```

```
        ? Math.max(...productos.map(p => p.id_producto)) + 1
```

```
        : 1;
```

```
}
```

actualizarProducto(id, ...): Acepta un objeto Partial<Producto> (para la consola) o parámetros individuales (para el router HTTP).

eliminarProducto(id): Elimina y devuelve texto de confirmación. Si no existe, lanza NotFoundError.

5.2 Funciones del clientesService

El servicio de clientes es un espejo del de productos con las mismas funciones pero para el modelo Cliente: listarClientes(), buscarCliente(), crearCliente(), agregarCliente(), actualizarCliente() y eliminarCliente().

Código:

productoService:

```
1 import { Producto } from "../models/producto";
2 import { ProductoRepository } from "../data/productoRepository";
3 import { validarProducto } from "../validator/validations";
4 import { NotFoundError } from "../errors/customErrors";
5
6 const repository = new ProductoRepository();
7
8 export async function listarProducto(): Promise<Producto[] | string> {
9   return repository.obtenerTodos();
10 }
11
12 export async function listarProductos(): Promise<Producto[]> {
13   return repository.obtenerTodos();
14 }
15
16 export async function buscarProducto(id: number): Promise<Producto | null> {
17   const p = await repository.buscarPorId(id);
18   if (!p) {
19     throw new NotFoundError(`No se encontró el producto con ID ${id}.`);
20   }
21   return p;
22 }
23
24 export async function crearProducto(nuevoProducto: Producto): Promise<void> {
25   validarProducto(nuevoProducto);
26   const productos = await repository.obtenerTodos();
27   nuevoProducto.id_producto = productos.length > 0 ? Math.max(...productos.map(p => p.id_producto)) + 1 : 1;
28   productos.push(nuevoProducto);
29   await repository.guardar(productos);
30 }
31
32 export async function agregarProducto(
33   id: number,
34   nombre: string,
35   precio: number,
36   stock: number,
37   categoria: any,
38   estado: any,
39   descuento: number
40 ): Promise<Producto | string> {
41   const productos = await repository.obtenerTodos();
42
43   let idAutoIncrement = id;
44   if (id === undefined || id === null || Number.isNaN(id) || id <= 0) {
45     idAutoIncrement = productos.length > 0 ? Math.max(...productos.map(p => p.id_producto)) + 1 : 1;
46   } else {
47     if (productos.some(p => p.id_producto === id)) {
```

```

48         return `El ID del producto ${id} ya existe.`;
49     }
50 }
51
52 const nuevo: Producto = {
53     id_producto: idAutoIncrement,
54     nombre_producto: nombre,
55     codigo_producto: Math.floor(1000 + Math.random() * 9000),
56     cantidad: stock,
57     precio: precio,
58     categoria: categoria,
59     estado_venta: estado,
60     descuento: descuento
61 };
62
63 validarProducto(nuevo);
64
65 productos.push(nuevo);
66 await repository.guardar(productos);
67 return nuevo;
68 }
69
70 export async function actualizarProducto(
71     id: number,
72     nombreOrDatos: string | Partial<Producto>,
73     precio?: number,
74     stock?: number,
75     categoria?: any,
76     estado?: any,
77     descuento?: number
78 ): Promise<boolean | Producto> {
79     const productos = await repository.obtenerTodos();
80     if (id <= 0) {
81         throw new NotFoundError("El ID del producto debe ser mayor que 0.");
82     }
83
84     const index = productos.findIndex(p => p.id_producto === id);
85     if (index === -1) {
86         throw new NotFoundError(`No se encontró el producto con ID ${id}.`);
87     }
88
89     let actualizarDatos: Partial<Producto> = {};
90
91     if (typeof nombreOrDatos === "object" && nombreOrDatos !== null) {
92         actualizarDatos = nombreOrDatos;
93     } else {
94         if (nombreOrDatos !== undefined) actualizarDatos.nombre_producto = nombreOrDatos;
95         if (precio !== undefined) actualizarDatos.precio = precio;
96         if (stock !== undefined) actualizarDatos.cantidad = stock;
97         if (categoria !== undefined) actualizarDatos.categoria = categoria;

```

```

97     if (categoria !== undefined) actualizarDatos.categoria = categoria;
98     if (estado !== undefined) actualizarDatos.estado_venta = estado;
99     if (descuento !== undefined) actualizarDatos.descuento = descuento;
100 }
101
102 productos[index] = { ...productos[index], ...actualizarDatos };
103 await repository.guardar(productos);
104
105 if (typeof nombreOrDatos === "object" && nombreOrDatos !== null) {
106     return true;
107 } else {
108     return productos[index];
109 }
110 }
111
112 export async function eliminarProductoPorId(id: number): Promise<boolean> {
113     const productos = await repository.obtenerTodos();
114     if (id <= 0) return false;
115
116     const index = productos.findIndex(p => p.id_producto === id);
117     if (index === -1) {
118         return false;
119     }
120
121     productos.splice(index, 1);
122     await repository.guardar(productos);
123     return true;
124 }
125
126 export async function eliminarProducto(id: number): Promise<string> {
127     const result = await eliminarProductoPorId(id);
128     if (!result) {
129         throw new NotFoundError(`No se encontró el producto con ID ${id}.`);
130     }
131     return "Producto eliminado correctamente.";
132 }
133
134 export const calcularSubtotal = (montos: number[]): number =>
135     montos.reduce((acumulado, monto) => acumulado + monto, 0);
136
137 export const calcularIVA = (subtotal: number, tasaIVA: number): number =>
138     subtotal * tasaIVA;
139
140 export const calcularTotalFinal = (subtotal: number, iva: number): number =>
141     subtotal + iva;
142
143 export async function calcularVentaProducto(id: number, IVA: number = 0.12): Promise<void> {
144     const productos = await repository.obtenerTodos();
145     const producto = productos.find(p => p.id_producto === id);

```

```

146     const producto = productos.find(p => p.id_producto === id);
147
148     if (!producto) {
149         console.log("El producto no existe.");
150         return;
151     }
152
153     const subtotal = calcularSubtotal([producto.precio]);
154     const iva = calcularIVA(subtotal, IVA);
155     const total = calcularTotalFinal(subtotal, iva);
156
157     console.log("|----- resumen de venta -----|");
158     console.log("-- ID: " + producto.id_producto);
159     console.log("-- subtotal: " + subtotal);
160     console.log("-- iva: " + iva);
161     console.log("-- total: " + total);
162 }

```

6. El Servidor HTTP (server.ts)

Archivo: src/api/server.ts

El servidor es quien crea la instancia HTTP de Node.js y la pone a escuchar en el puerto 3000. Se modificó para exportar la instancia del servidor:

A screenshot of a code editor with a dark background and light-colored text. The code is written in TypeScript and defines an HTTP server. It imports the 'http' module and a 'router' from a local file. It sets a port of 3000 and creates an async server that uses the router. The server is then listened to on the specified port, with console logs indicating the server is running at http://localhost:3000.

```
1 import * as http from "http";
2 import { router } from "../router";
3
4 const port = 3000;
5
6 export const server = http.createServer(async (req, res) => {
7   await router(req, res);
8 });
9
10 server.listen(port, () => {
11   console.log("-----")
12   console.log("Servidor en http://localhost:" + port);
13   console.log("-----")
14 });
```

El motivo de exportarlo es permitirle al archivo de entrada (index.ts) esperar a que el servidor esté completamente listo antes de mostrar el menú de consola, evitando que los mensajes aparezcan en el orden incorrecto.

7. El Enrutador (router.ts)

Archivo: src/api/router.ts

Este es el archivo más grande y el corazón de la API. Es quien recibe cada petición HTTP y decide qué hacer con ella según la URL y el método (GET, POST, PUT, DELETE).

7.1 La función enviarError

Centraliza toda la lógica de respuesta de errores. Detecta el tipo de error y asigna el código HTTP correcto (ValidationError → 400, NotFoundError → 404):



```
1  function enviarError(res: http.ServerResponse, error: any) {
2      let errorType = "UnknownError";
3      let statusCode = 400;
4      let message = "Ocurrió un error inesperado.";
5
6      if (error instanceof Error) {
7          errorType = error.name;
8          message = error.message;
9          if (errorType === "NotFoundError") {
10             statusCode = 404;
11         }
12     } else if (typeof error === "string") {
13         errorType = "ValidationError";
14         message = error;
15     }
16
17     res.writeHead(statusCode, {
18         "Content-Type": "application/json"
19     });
20     res.end(JSON.stringify({ errorType, message }));
21 }
```

7.2 Endpoints de Productos

Método	URL	Descripción
GET	/productos	Lista todos los productos
POST	/productos	Crea un nuevo producto (ID auto-incrementado)
GET	/productos/:id	Busca un producto por ID
PUT	/productos/:id	Actualiza campos de un producto

DELETE /productos/:id Elimina un producto

7.3 Endpoints de Clientes

Método URL Descripción

GET /clientes Lista todos los clientes

POST /clientes Crea un nuevo cliente (ID auto-incrementado)

GET /clientes/:id Busca un cliente por ID

PUT /clientes/:id Actualiza campos de un cliente

DELETE /clientes/:id Elimina un cliente

Código de los endpoints:

Endpoints clientes:

```
1  export async function router(req : http.IncomingMessage, res : http.ServerResponse){
2
3      try {
4          const partes = req.url?.split("/");
5
6          // /**----- endpoints para productos -----**/
7          if(req.url === "/productos" && req.method === "GET"){
8              res.writeHead(200, {
9                  "Content-Type": "application/json"
10             });
11             res.end(JSON.stringify(await listarProductos()));
12             return;
13         }
14
15         if(req.url === "/productos" && req.method === "POST"){
16             let body = "";
17
18             req.on("data", (chunk) =>{
19                 body += chunk;
20             });
21
22             req.on("end", async () =>{
23                 try {
24                     const p = JSON.parse(body);
25
26                     const id = p.id_producto !== undefined ? p.id_producto : p.id;
27                     const nombre = p.nombre_producto || p.nombre;
28                     const stock = p.cantidad !== undefined ? p.cantidad : p.stock;
29                     const estado = p.estado_venta || p.estado;
30                     const codigo = p.codigo_producto !== undefined ? p.codigo_producto : p.codigo;
31
32                     const result = await (agregarProducto as any)(
33                         Number(id),
34                         nombre,
35                         Number(p.precio),
36                         Number(stock),
37                         p.categoria,
38                         estado,
39                         Number(p.descuento),
40                         codigo !== undefined ? Number(codigo) : 0
41                     );
42
43                     if (typeof result === "string") {
44                         enviarError(res, result);
45                         return;
46                     }
47                 }
48             });
49         }
50     }
51 }
```

```

48         res.writeHead(201, {
49             "Content-Type": "application/json"
50         });
51         res.end(JSON.stringify(result));
52     } catch (error) {
53         enviarError(res, error);
54     }
55 });
56 return;
57 }
58
59 if(partes?.length === 3 && partes[1] === "productos" && req.method === "GET"){
60     const id = Number(partes[2]);
61     const p = await buscarProducto(id);
62
63     if(!p){
64         res.writeHead(404, {
65             "Content-Type": "application/json"
66         });
67         res.end(JSON.stringify({ error : "El id no existe"}));
68         return;
69     }
70
71     res.writeHead(200,{
72         "Content-Type":"application/json"
73     });
74     res.end(JSON.stringify(p));
75     return;
76 }
77
78 if(partes?.length === 3 && partes[1] === "productos" && req.method === "PUT"){
79     const id = Number(partes[2]);
80     let body = "";
81
82     req.on("data", (chunk) =>{
83         body += chunk;
84     });
85
86     req.on("end", async () =>{
87         try {
88             const p = JSON.parse(body);
89             const result = await actualizarProducto(id, p.nombre, p.precio, p.stock, p.categoria, p.estado, p.descuento);
90
91             res.writeHead(200,{
92                 "Content-Type":"application/json"
93             });
94             res.end(JSON.stringify(result));
95         } catch (error) {
96             enviarError(res, error);

```

```

96             enviarError(res, error);
97         }
98     });
99     return;
100 }
101
102 if(partes?.length === 3 && partes[1] === "productos" && req.method === "DELETE"){
103     const id = Number(partes[2]);
104     const result = await eliminarProducto(id);
105
106     res.writeHead(200, {
107         "Content-Type": "text/plain"
108     });
109     res.end(result);
110     return;
111 }

```

7.4 Try/Catch global

Todo el cuerpo del router está envuelto en un try/catch global. Si ocurre cualquier error inesperado, este lo atrapa y devuelve una respuesta estructurada en lugar de dejar caer el servidor.

8. El Punto de Entrada (index.ts)

Archivo: src/index.ts

En su versión final hace dos cosas: importar el servidor (lo que lo arranca automáticamente) y esperar a que confirme que está listo para mostrar el menú:

A screenshot of a code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is written in TypeScript and is as follows:

```
1 import { server } from "../api/server";
2 import { menu } from "../menu/menu";
3
4 server.on("listening", () => {
5   menu();
6 });
```

Con esto se garantiza el orden correcto: primero aparece el mensaje del servidor con el puerto, luego el menú interactivo. Ambos corren simultáneamente en el mismo proceso de Node.js.

9. Flujo de una petición de inicio a fin

Ejemplo: POST /productos desde Postman:

El usuario hace la petición al puerto 3000.

server.ts la recibe y se la pasa al router.

El router identifica que es un POST /productos y lee el cuerpo JSON.

El router llama a agregarProducto() del productoService.

El service llama a repository.obtenerTodos() para verificar que el ID no exista.

El repository usa leerProductos() para leer el archivo producto.json.

El service crea el objeto Producto y llama a validarProducto().

Si pasa validación, llama a `repository.guardar()` que escribe en `producto.json`.

El service devuelve el objeto creado al router.

El router responde con código 201 y el JSON del producto creado.

Si en cualquier paso ocurre un error, se lanza la excepción correspondiente, el `try/catch` del router la captura, y `enviarError()` responde con el código correcto.

10. Formato de las Respuestas

Respuesta exitosa (201 Created):

JSON

```
{
  "id_producto": 7,
  "nombre_producto": "Martillo",
  "codigo_producto": 3542,
  "cantidad": 15,
  "precio": 75,
  "categoria": "de hogar",
  "estado_venta": "pagado",
  "descuento": 5
}
```

Error de validación (400):

JSON:

```
{  
  "errorType": "error de validacion",  
  "message": "El nombre no puede estar vacío."  
}
```

Recurso no encontrado (404):

JSON:

```
{  
  "errorType": "Error no encontrado",  
  "message": "No se encontró el producto con ID 999."  
}
```

Cabecera de tiempo de respuesta (visible en Postman → Headers):

<code>src/data/clienteRepository.ts</code>	NUEVO	Repositorio de persistencia de clientes
<code>src/data/productoRepository.ts</code>	NUEVO	Repositorio de persistencia de productos
<code>src/validator/validaciones.ts</code>	MODIFICADO	Lanza <code>ValidationError</code> en lugar de <code>Error</code> genérico
<code>src/services/clientesService.ts</code>	MODIFICADO	Usa repositorio, ID auto-incrementado, errores personalizados
<code>src/services/productoService.ts</code>	MODIFICADO	Usa repositorio, ID auto-incrementado, errores personalizados
<code>src/api/router.ts</code>	MODIFICADO	CRUD completo, manejo de errores y tiempo de respuesta
<code>src/api/server.ts</code>	MODIFICADO	Exporta la instancia del servidor
<code>src/index.ts</code>	MODIFICADO	Arranca servidor y menú de forma concurrente y ordenada